



CITS5017 Deep Learning Semester 2, 2026

Lab Sheet 1 (Not assessed)

1. Outline

This labsheet is a good exercise to practise what you learned from the lecture notes for Topic 1 and Chapter 10 “*Introduction to Artificial Neural Networks*” of the textbook.

2. Tasks

2.1. Task 1

The first task for this week is to go through the Jupyter Notebook file for Chapter 10 supplied by Aurélien Géron on the GitHub page <https://github.com/ageron/handson-ml3>. This small task will help confirm that you have your Python environment set up properly for the unit if you run the code on your computer. This year, the [High Performance Computing \(HPC\)](#) will provide GPU access to CITS5017 students. If the set up is complete before the start of week 2 of the semester, then you should be able to transfer the Jupyter Notebook file for Chapter 10 to Kaya (the login node of the GPU computers) and run the code there.

You will need to make some minor modification to the Jupyter Notebook file from the author as some parameter/argument names (e.g., `input_shape`) used in his code are deprecated. For instance,

```
# Code from the author of the textbook would yield a User Warning message.
model = tf.keras.Sequential()
model.add(tf.keras.layers.Flatten(input_shape=[28, 28]))
...

# From approximately tf-2.12 onward, do the following instead:
model = tf.keras.Sequential()
model.add(tf.keras.layers.InputLayer(shape=[28, 28]))
model.add(tf.keras.layers.Flatten())
...
```

2.2. Task 2

A numpy data file named *circle.npy* can be found on the LMS page of the unit. Your task is to train two small MLPs for binary classification using the 2D synthesized data stored in this file. There are altogether 1,000 data points and the two classes are almost balanced. Download this data file. Then create a new Jupyter Notebook file and copy the following function to a code cell:

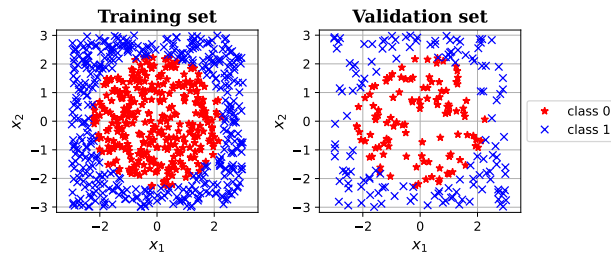


Figure 1: The ground truth training and validation sets in *circle.npy*.

```
import numpy as np
def load_data(filename):
    with open(filename, "rb") as f:
        xtrain = np.load(f)
        ytrain = np.load(f)
        xval = np.load(f)
        yval = np.load(f)
    return xtrain, ytrain, xval, yval
```

Call the function to create your training and validation sets as follows (assuming that you have saved *circle.npy* in the same directory where you started *jupyter-lab*):

```
X_train, y_train, X_val, y_val = load_data("circle.npy")
```

Write a small function to show the two classes in (X_{train}, y_{train}) and in (X_{val}, y_{val}) , like the diagram shown in Figure 1. Create a simple MLP that has one hidden layer of around 5 neurons. Train the model using the `fit` method for 100 epochs. You should pass (X_{val}, y_{val}) as validation data to the method. A simple optimizer like the Stochastic Gradient Descent (`sgd`) and the ReLU activation function should suffice for this simple dataset.

Repeat the above steps for a more complex MLP with two hidden layers. You can try 5 neurons for both hidden layers.

Compare the two MLPs on the following:

- their numbers of trainable parameters;
- their training and validation accuracies;
- their predicted outputs (visualization).

For the first model, you should be able to achieve a validation accuracy of 80% (or maybe a bit higher). For the second model, you should be able to get above 90% validation accuracy.

Obviously both models could be fine tuned to improve their performances even further, like using a more efficient optimizer, having a learning rate scheduler, and/or early stopping. You can try these strategies after we have covered Topic 2.

Do you think an MLP with just one hidden layer is sufficient for this dataset? Or do you think we need to have at least two hidden layers?

2.3. Task 3

Create a new Jupyter Notebook file for this task.

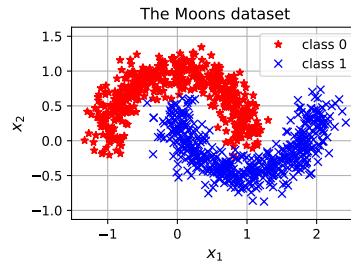


Figure 2: The moons dataset (having 1,000 points).

Use the `make_moons` function from the `sklearn.datasets` package to generate 1,000 data points (see Figure 2):

```
X, y = make_moons(n_samples=1000, noise=0.15, random_state=42)
```

The two classes in this dataset should be perfectly balanced. Use the `train_test_split` function from the `sklearn.model_selection` package to split the data, with a split ratio of 80/20, into a training set and a test set. For repeatability purpose, set the `random_state` to 42. Note that, for this task, there is not a validation set.

Do the following:

1. Train a Perceptron (with 1,000 iterations) using the training set and evaluate its accuracies on both the training set and the test set. Write a function (so that you can use it again later) to display the prediction results on the training set and the test set side-by-side.
2. Next, train an MLP with a single hidden layer of around 10 neurons using the training set. You can try the simple ReLU or the SELU activation function. After the model is trained, evaluate its performances on both the training set and on the test set. Call the function that you have written to display the prediction results graphically.

Did this MLP outperform the Perceptron model above?

3. Thirdly, try adding a second hidden layer of 5 neurons to the previous MLP to form a new MLP of two hidden layers. Repeat the training step as above. After this second MLP is trained, test its performance on both the training set and the test set. Again, display the prediction results graphically.

Did this new MLP outperform the Perceptron model above?

4. Finally, create a fourth model by copying the last MLP model above but without using any activation function. Repeat the training, testing, and visualization steps.

How did this model perform compared to the MLP model in subtask 3 above?

3. The TensorFlow Playground

The <http://playground.tensorflow.org> page is a useful learning tool for the MLP network. It has the MLP network implemented for the *XOR* problem, the *circle* problem (similar to Figure 1), etc. You can interactively design the network by changing the number of hidden layers, number of neurons per layer, activation function for each hidden layer, etc. For instance, by setting *activation* to *linear*¹, you should

¹Linear activation is equivalent to no activation.

see that the MLP becomes a linear classifier, no matter how many hidden layers there are in the network. For the *spiral* problem, using only two input features x_1 and x_2 would not be sufficient. You should include the three second-degree polynomial features x_1^2 , x_2^2 , and x_1x_2 as input to help. If you include the $\sin(x_1)$ and $\sin(x_2)$ features also, then the MLP may produce a reasonably good classification result with just one hidden layer of 5 neurons. Adding another hidden layer of around 5 neurons would improve the classification further. Try setting the ratio of training to test data to 80%, learning rate to 0.01, activation function to ReLU, and no regularization.